

Secure Multiplayer Lobby Protocol

Autor: Konrad Iwanczewski

Github: <https://github.com/AtelierRq/Secure-Multiplayer-Lobby-Protocol>

Etap 3 — Dokumentacja uruchomienia, scenariusze oraz rezultaty implementacji.

1. Dokumentacja uruchomienia

1.1. Instrukcja uruchomienia

Uruchomienie serwera

1. Przejdź do katalogu głównego projektu.
2. Wykonaj komendę w celu wygenerowania certyfikatów:
`powershell -ExecutionPolicy Bypass -File .\generate_certs.ps1`
3. Upewnij się, że certyfikaty znajdują się w katalogu:
`certs/`
4. Kompilacja:
`mingw32-make`
3. Uruchom serwer:
`bin\smlp_server.exe 1234`

gdzie:

- 1234 – numer portu nasłuchiwania.

Po poprawnym uruchomieniu powinien pojawić się komunikat:

```
[SMLP] Server listening on port 1234
```

Uruchomienie klienta

Uruchom klienta podając adres serwera oraz port:

```
bin\smlp_client.exe 127.0.0.1 1234
```

gdzie:

- 127.0.0.1 – adres serwera,
- 1234 – numer portu.

Po nawiązaniu połączenia klient poprosi o podanie nazwy użytkownika (nickname).

Zakończenie działania klienta

Aby zakończyć działanie klienta należy wpisać:

```
exit
```

lub zamknąć okno terminala.

Zakończenie działania serwera

Zamknięcie okna serwera powoduje zakończenie działania aplikacji i rozłączenie wszystkich klientów.

1.2. Wymagania środowiskowe

System operacyjny

Projekt został przygotowany i przetestowany w środowisku:

Windows 10 / Windows 11

Kompilator

MinGW-w64 GCC

Przykładowa wersja:

gcc 14.x

Biblioteki

Projekt wymaga bibliotek:

OpenSSL

Winsock2

Wykorzystywane moduły:

- libssl
- libcrypto

Certyfikaty

Przed uruchomieniem należy wygenerować certyfikaty TLS za pomocą przygotowanego skryptu PowerShell.

Wszystkie certyfikaty muszą znajdować się w katalogu:

certs/

1.3. Komendy protokołu SMLP

Logowanie

LOGIN|<nickname>

Przykład:

LOGIN|gracz1

Utworzenie lobby

CREATE_LOBBY|<nazwa_lobby>

Przykład:

CREATE_LOBBY|Room1

Dołączenie do lobby

JOIN|<nazwa_lobby>

Przykład:

JOIN|Room1

Opuszczenie lobby

LEAVE

Wyświetlenie dostępnych lobby

LIST_LOBBIES

Przykładowa odpowiedź:

LOBBIES|Room1(2),Room2(1)

Wyświetlenie graczy w aktualnym lobby

LIST_PLAYERS

Przykładowa odpowiedź:

PLAYERS|gracz1[Host],gracz2

Oznaczenie gotowości

READY

Przykładowa odpowiedź:

READY_OK

Rozpoczęcie gry (tylko host)

START

Przykładowa odpowiedź:

GAME_STARTED

Zakończenie gry

END_GAME

Przykładowa odpowiedź:

GAME_ENDED

Wiadomość czatu

CHAT|<wiadomość>

Przykład:

CHAT|witajcie

Przykładowa odpowiedź:

CHAT_MSG|gracz1|witajcie

Wyjście z aplikacji klienckiej

exit

1.4. Znane ograniczenia

1. Projekt wykorzystuje architekturę klient-serwer i pojedynczy serwer centralny.
2. W przypadku rozłączenia hosta lobby jest usuwane.
3. Timeout klienta oparty jest na czasie nieaktywności wynoszącym 120 sekund.

4. Projekt nie implementuje mechanizmu transferu hosta pomiędzy graczami.
5. Projekt nie implementuje trwałych kont użytkowników i haseł.
6. Autoryzacja odbywa się wyłącznie na podstawie unikalnego nickname.

Test 1 – Pełny poprawny scenariusz

Cel

Sprawdzenie pełnego przebiegu sesji.

Kroki

1. Uruchom serwer.

```
bin\smlp_server.exe 1234  
[SMLP] Server listening on port 1234  
█
```

2. Uruchom klienta A.

```
bin\smlp_client.exe 127.0.0.1 1234  
  
[TLS] Handshake successful  
[TLS] Certificate verification successful  
  
[TLS] Server certificate:  
Subject: /C=PL/O=SMLP/CN=127.0.0.1  
Issuer : /C=PL/O=SMLP/CN=SMLP-CA  
Nickname: █
```

3. Login jako:
gracz1

```
Nickname: gracz1  
  
[SMLP] Server response:  
LOGIN_OK|TOKEN_1_1780424573  
  
[SMLP] Login successful  
  
SMLP[gracz1]> █
```

4. Utwórz lobby:
CREATE_LOBBY|Room1

```
SMLP[gracz1]> CREATE_LOBBY|Room1

SMLP[gracz1]>
SERVER: LOBBY_CREATED|1|Room1

SMLP[gracz1|Room1]> █
```

5. Uruchom klienta B.

```
bin\smlp_client.exe 127.0.0.1 1234

[TLS] Handshake successful
[TLS] Certificate verification successful

[TLS] Server certificate:
Subject: /C=PL/O=SMLP/CN=127.0.0.1
Issuer : /C=PL/O=SMLP/CN=SMLP-CA
Nickname: █
```

6. Login jako:
gracz2

```
Nickname: gracz2

[SMLP] Server response:
LOGIN_OK|TOKEN_2_1780424755

[SMLP] Login successful

SMLP[gracz2]> █
```

7. Dołącz:
JOIN|Room1

```
SMLP[gracz2]> JOIN|Room1

SMLP[gracz2]>
SERVER: JOIN_OK|Room1

SMLP[gracz2|Room1]> █
```

8. Wykonaj:
CHAT|Cześć

```
SMLP[gracz2|Room1]> CHAT|Cześć  
  
SMLP[gracz2|Room1]>  
[gracz2]: Cześć  
  
SMLP[gracz2|Room1]> █
```

```
SMLP[gracz1|Room1]>  
[gracz2]: Cześć
```

9. Obaj gracze:
READY

```
SMLP[gracz2|Room1]> READY  
  
SMLP[gracz2|Room1]>  
SERVER: READY_OK  
  
SMLP[gracz2|Room1|READY]> █
```

```
SMLP[gracz1|Room1]> READY  
  
SMLP[gracz1|Room1]>  
SERVER: READY_OK  
  
SMLP[gracz1|Room1|READY]> █
```

10. Host:
START

```
SERVER: GAME_STARTED  
  
SMLP[gracz2|Room1|IN_GAME]> █
```

```
SMLP[gracz1|Room1|READY]> START  
  
SMLP[gracz1|Room1|READY]>  
SERVER: GAME_STARTED  
  
SMLP[gracz1|Room1|IN_GAME]> █
```

11. Host:
END_GAME

```
SERVER: GAME_ENDED  
  
SMLP[gracz2|Room1]> █
```



```
SMLP[gracz1|Room1|IN_GAME]> END_GAME  
  
SMLP[gracz1|Room1|IN_GAME]>  
SERVER: GAME_ENDED  
  
SMLP[gracz1|Room1]> █
```

12. Obaj gracze:

LEAVE

```
SMLP[gracz2|Room1]> LEAVE  
  
SMLP[gracz2|Room1]>  
SERVER: LEAVE_OK  
  
SMLP[gracz2]> █
```

```
SMLP[gracz1|Room1]> LEAVE  
  
SMLP[gracz1|Room1]>  
SERVER: LEAVE_OK  
  
SMLP[gracz1]> █
```

Oczekiwany wynik

LOGIN_OK
LOBBY_CREATED
JOIN_OK
CHAT_MSG
READY_OK
GAME_STARTED
GAME_ENDED
LEAVE_OK

```
[SMLP] Server listening on port 1234
[INFO] Client connected (id=1)
[TLS] Handshake successful (client id=1)
[RECV][1] LOGIN|gracz1
[INFO] User logged in: gracz1
[RECV][1] CREATE_LOBBY|Room1
[LOBBY] gracz1 created lobby Room1
[INFO] Client connected (id=2)
[TLS] Handshake successful (client id=2)
[RECV][2] LOGIN|gracz2
[INFO] User logged in: gracz2
[RECV][2] JOIN|Room1
[LOBBY] gracz2 joined Room1
[RECV][2] CHAT|Cześć
[CHAT][gracz2] Cześć
[RECV][2] READY
[READY] gracz2 is ready
[RECV][1] READY
[READY] gracz1 is ready
[RECV][1] START
[GAME] Lobby 1 started
[RECV][1] END_GAME
[GAME] Lobby 1 ended
[RECV][2] LEAVE
[RECV][1] LEAVE
[LOBBY] Lobby Room1 removed by host
█
```

Status:

PASS

Test 2 – Duplikat nicku

Cel

Sprawdzenie unikalności nicków.

Kroki

Klient 1:

LOGIN gracz1

```
Nickname: gracz1

[SMLP] Server response:
LOGIN_OK|TOKEN_1_1780425559

[SMLP] Login successful

SMLP[gracz1]> █
```

Klient 2:

LOGIN gracz1

```
[TLS] Server certificate:
Subject: /C=PL/O=SMLP/CN=127.0.0.1
Issuer : /C=PL/O=SMLP/CN=SMLP-CA
Nickname: gracz1

[SMLP] Server response:
ERROR|Nickname already used

Nickname is unavailable. Try again.

Nickname: █
```

Oczekiwany wynik

ERROR|Nickname already used

oraz ponowne pytanie o nick.

Status:

PASS

Test 3 – Nie-host uruchamia grę

Kroki

Host:

CREATE_LOBBY|Room1

```
SMLP[gracz1]> CREATE_LOBBY|Room1

SMLP[gracz1]>
SERVER: LOBBY_CREATED|1|Room1

SMLP[gracz1|Room1]> █
```

Gracz2:

JOIN|Room1

```
SMLP[gracz2]> JOIN|Room1  
  
SMLP[gracz2]>  
SERVER: JOIN_OK|Room1  
  
SMLP[gracz2|Room1]> █
```

Gracz2:

START

```
SMLP[gracz2|Room1]> START  
  
SMLP[gracz2|Room1]>  
SERVER: ERROR|Only host can start game  
  
SMLP[gracz2|Room1]> █
```

Oczekiwany wynik

ERROR|Only host can start game

Status:

PASS

Test 4 – Start bez READY

Kroki

Host:

CREATE_LOBBY|Room1

```
SMLP[gracz1]> CREATE_LOBBY|Room1  
  
SMLP[gracz1]>  
SERVER: LOBBY_CREATED|1|Room1
```

Gracz2:

JOIN|Room1

```
SMLP[gracz2]> JOIN|Room1
```

```
SMLP[gracz2]>  
SERVER: JOIN_OK|Room1
```

Host:

START

```
SMLP[gracz1|Room1]> START  
  
SMLP[gracz1|Room1]>  
SERVER: ERROR|Not all players ready  
  
SMLP[gracz1|Room1]> █
```

Oczekiwany wynik

ERROR|Not all players ready

Status:

PASS

Test 5 – Krótki test obciążeniowy

Kroki

Uruchom:

5 klientów

```
[SMLP] Login successful  
  
SMLP[gracz1]> █
```

```
[SMLP] Login successful  
  
SMLP[gracz2]> █
```

```
[SMLP] Login successful  
  
SMLP[gracz3]> █
```

```
[SMLP] Login successful
```

```
SMLP[gracz4]> 
```

```
[SMLP] Login successful
```

```
SMLP[gracz5]> 
```

Wszyscy:

JOIN|Room1

```
SERVER: LOBBY_CREATED|1|Room1
```

```
SMLP[gracz1|Room1]> LIST_PLAYERS
```

```
SMLP[gracz1|Room1]>
```

```
SERVER: PLAYERS|gracz1[Host],gracz2,gracz3,gracz4,gracz5,
```

```
SMLP[gracz1|Room1]> 
```

CHAT|test

```
SMLP[gracz1|Room1]> CHAT|test
```

```
SMLP[gracz1|Room1]>
```

```
[gracz1]: test
```

```
SMLP[gracz1|Room1]>
```

```
[gracz2]: Test2
```

```
SMLP[gracz1|Room1]>
```

```
[gracz3]: test3
```

```
SMLP[gracz1|Room1]>
```

```
[gracz4]: test4
```

```
SMLP[gracz1|Room1]>
```

```
[gracz5]: test5
```

```
SMLP[gracz1|Room1]> 
```

READY

```
[RECV][1] READY
[READY] gracz1 is ready
[RECV][2] READY
[READY] gracz2 is ready
[RECV][3] READY
[READY] gracz3 is ready
[RECV][4] READY
[READY] gracz4 is ready
[RECV][5] READY
[READY] gracz5 is ready
█
```

Oczekiwany wynik

Brak:

crash

deadlock

utrata połączenia

Status:

PASS

Podsumowanie

W ramach projektu udało się zaimplementować wszystkie kluczowe elementy protokołu SMLP (Secure Multiplayer Lobby Protocol):

- komunikację klient-serwer opartą o protokół TCP,
- szyfrowanie komunikacji przy użyciu TLS (OpenSSL),
- uwierzytelnianie użytkowników za pomocą unikalnych nickname'ów,
- walidację unikalności nazw użytkowników,
- tworzenie lobby przez użytkownika pełniącego rolę hosta,
- dołączanie do istniejących lobby,
- opuszczanie lobby,
- automatyczne usuwanie lobby po opuszczeniu go przez hosta,
- wyświetlanie listy dostępnych lobby,
- wyświetlanie listy graczy znajdujących się w lobby,
- oznaczanie gotowości graczy (READY),
- rozpoczynanie sesji gry przez hosta (START),
- kończenie sesji gry (END_GAME),
- komunikację tekstową pomiędzy uczestnikami lobby (CHAT),

- rozsyłanie wiadomości do wszystkich uczestników lobby (broadcast),
- asynchroniczny odbiór komunikatów po stronie klienta,
- obsługę błędów protokołu i niepoprawnych komunikatów,
- obsługę rozłączeń klientów,
- mechanizm timeout rozłączający nieaktywnych użytkowników,
- rejestrowanie zdarzeń serwera w pliku logów,
- podstawowe testy funkcjonalne, błędów i obciążeniowe.